CASE STUDY REPORT

Traditional vs Low-Code Development

A Technology Selection Study for a

College Management Application

An evidence-based comparison of hand-coded software engineering and modern low-code application platforms, evaluated against real delivery timelines, budgets, and long-term ownership costs for a nine-module college administration system.

DOCUMENT TYPE	Technology Comparison Case Study
APPLICATION DOMAIN	Higher Education Administration (EdTech)
MODULES IN SCOPE	9 core modules — admissions to reporting
COMPARISON SCOPE	Traditional Engineering vs. Low-Code Platforms
INTENDED USE	Technology strategy & vendor / stack selection

NAME
Keerthana V

ROLL NO.
241501082

Table of Contents

1. Executive Summary . . . 3

2. Case Study Overview . . . 3

3. Project Requirements . . . 3

4. Development Approaches . . . 5

4.1 Platform & Stack Notes . . . 5

5. Case Study Comparison Matrix . . . 5

6. Development Process Comparison . . . 7

6.1 Traditional Development Process . . . 7

6.2 Low-Code Development Process . . . 7

7. Estimated Timelines . . . 7

7.1 Traditional Development Timeline . . . 7

7.2 Low-Code Development Timeline . . . 8

7.3 Timeline Comparison . . . 8

8. Cost Comparison . . . 9

8.1 Total Cost of Ownership (3-Year View) . . . 9

9. Risk Analysis . . . 9

10. Advantages & Disadvantages . . . 11

10.1 Traditional Development . . . 11

10.2 Low-Code Development . . . 11

11. Results of the Case Study . . . 11

12. Decision Framework . . . 12

13. Conclusion & Recommendation . . . 12

1. Executive Summary

Educational institutions are under increasing pressure to digitise administrative operations — admissions, attendance, examinations, fees, and library services — while controlling budgets and delivery timelines. This report compares two ways of building a College Management Application: **Traditional Development**, using hand-written code on stacks such as Java, Python, or .NET, and **Low-Code Development**, using visual application platforms such as Microsoft Power Apps, OutSystems, Mendix, or Zoho Creator.

Across every measured dimension — time, team size, cost, and maintainability — the low-code approach delivers the same nine-module system in roughly a third of the time and at less than half the cost of traditional development. Traditional development, in turn, remains superior wherever the institution needs deep customisation, complex integrations, or full ownership of the source code and infrastructure.

The recommendation in Section 12 is scenario-based rather than absolute: institutions seeking a fast, standard administrative system should choose low-code; institutions with unique workflows, heavy integration needs, or long-horizon scalability requirements should choose traditional development.

2. Case Study Overview

Project: College Management Application

Purpose: To develop a web-based application that manages student admissions, faculty records, attendance, examinations, fees, and library operations for an educational institution, replacing fragmented spreadsheets and paper registers with a single connected system.

This case study evaluates two contrasting approaches to building the application — **Traditional Development** using hand-written code, and **Low-Code Development** using visual application platforms — and compares them across development time, cost, team effort, scalability, and long-term maintainability. The findings are intended to support IT leadership and college administration in selecting the most suitable delivery model before committing budget or vendor contracts.

3. Project Requirements

The application is expected to include the following nine functional modules. Each module maps to a distinct administrative function and carries its own data, workflow, and access-control requirements.

Module	Core Functionality
Student Registration	Application intake, document verification, enrolment confirmation, and ID generation.
Faculty Management	Faculty profiles, qualifications, subject allocation, and workload tracking.
Course Management	Program, course, syllabus, and credit-structure definition across departments.
Attendance Management	Daily and period-wise attendance capture with automated shortage alerts.
Examination & Results	Exam scheduling, marks entry, grade/GPA computation, and result publication.
Fee Management	Fee structure setup, invoicing, online payment collection, and defaulter tracking.
Library Management	Book cataloguing, issue/return workflow, and fine calculation.

Module	Core Functionality
User Login & Roles	Role-based access for administrators, faculty, students, and parents.
Reports & Dashboard	Real-time analytics and administrative reports for decision-making.

Table 3.1 — Functional modules and their core administrative purpose.

4. Development Approaches

Both approaches can technically deliver all nine modules; they differ in how the system is built, who can build it, and how much is produced by hand versus configured through a platform.

Traditional Development	Low-Code Development
Built with Java / Python / .NET on the backend, with HTML, CSS, JavaScript, and SQL for the front end and database layer.	Built on visual application platforms such as Microsoft Power Apps, OutSystems, Mendix, or Zoho Creator.
Every screen, workflow, and API endpoint is manually coded and version-controlled.	Screens, workflows, and data models are assembled through drag-and-drop designers and visual logic builders.
Requires experienced software developers familiar with the chosen stack, frameworks, and DevOps tooling.	Can be built by professional developers and, for simpler workflows, by trained business users (“citizen developers”).

Table 4.1 — How the two approaches differ in construction and skill requirements.

4.1 Platform & Stack Notes

- **Java / Python / .NET stacks** give full control over architecture, security hardening, and performance tuning — well suited to a system that must scale to tens of thousands of students across multiple campuses.
- **Microsoft Power Apps** integrates tightly with Microsoft 365 and Azure AD, making it a natural fit for colleges already on Microsoft infrastructure.
- **OutSystems and Mendix** are enterprise-grade low-code platforms with strong support for complex business logic, custom code extensions, and on-premise deployment.
- **Zoho Creator** is a lightweight, cost-effective option well suited to smaller institutions that need a working system quickly without heavy IT overhead.

5. Case Study Comparison Matrix

Parameter	Traditional Development	Low-Code Development
Development Time	5–6 months	1–2 months
Team Size	6–8 developers	2–4 developers
Coding Effort	High	Low
Cost	High	Moderate
UI Design	Manual	Pre-built templates
Database Integration	Manual coding	Built-in connectors
Testing	Manual and automated	Mostly automated
Deployment	Complex	One-click deployment
Maintenance	Time-consuming	Easier

Parameter	Traditional Development	Low-Code Development
Scalability	Very High	High (platform-dependent)
Customization	Unlimited	Limited by platform capabilities
Security	Fully controlled by developers	Platform-managed security

Table 5.1 — Parameter-by-parameter comparison across twelve delivery dimensions.

The matrix shows a consistent pattern: low-code wins decisively on speed, team size, and operational simplicity, while traditional development retains the edge wherever unlimited customisation or full architectural control is required. Security is not inherently weaker on either side — traditional puts the burden of correctness on the development team, while low-code delegates baseline security to the platform vendor.

6. Development Process Comparison

The two approaches follow structurally similar phases — analysis, design, build, test, deploy — but low-code compresses design and build into fewer, more visual steps because the platform generates boilerplate code, UI scaffolding, and database bindings automatically.

6.1 Traditional Development Process

1. Requirement Analysis
2. System Design
3. Database Design
4. UI Development
5. Backend Coding
6. API Development
7. Testing
8. Bug Fixing
9. Deployment
10. Maintenance

6.2 Low-Code Development Process

1. Requirement Analysis
2. Visual Workflow Design
3. Drag-and-Drop UI Creation
4. Configure Database
5. Integrate Services
6. Automated Testing
7. One-Click Deployment

Because low-code platforms auto-generate the data layer and much of the UI, three traditionally separate phases — database design, backend coding, and API development — collapse into a single “configure database” and “integrate services” step, which is the single biggest driver of the time savings shown in Section 7.

7. Estimated Timelines

7.1 Traditional Development Timeline

Phase	Duration
Requirement Analysis	2 weeks
Design	3 weeks
Coding	8 weeks
Testing	4 weeks
Deployment	1 week
Total	18 weeks (≈4.5–6 months)

7.2 Low-Code Development Timeline

Phase	Duration
Requirement Analysis	1 week
UI Design	1 week
Workflow Configuration	2 weeks
Testing	1 week
Deployment	2 days
Total	5–6 weeks (≈1–2 months)

7.3 Timeline Comparison

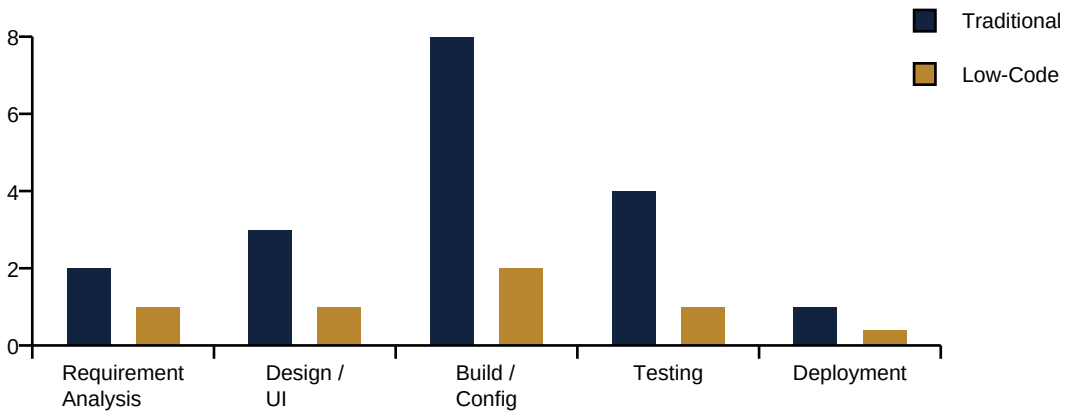

Figure 7.1 — Phase-wise duration in weeks, Traditional vs. Low-Code.

The gap is widest in the build phase, where traditional coding takes roughly four times longer than low-code configuration — the single largest contributor to the overall 3–4× speed advantage of the low-code approach.

8. Cost Comparison

Expense	Traditional	Low-Code
Development Cost	■8–12 Lakhs	■3–5 Lakhs
Maintenance Cost	High	Low
Infrastructure Cost	Medium	Low (Cloud-based)
Training Cost	High	Moderate

Table 8.1 — One-time and recurring cost categories (indicative, INR).

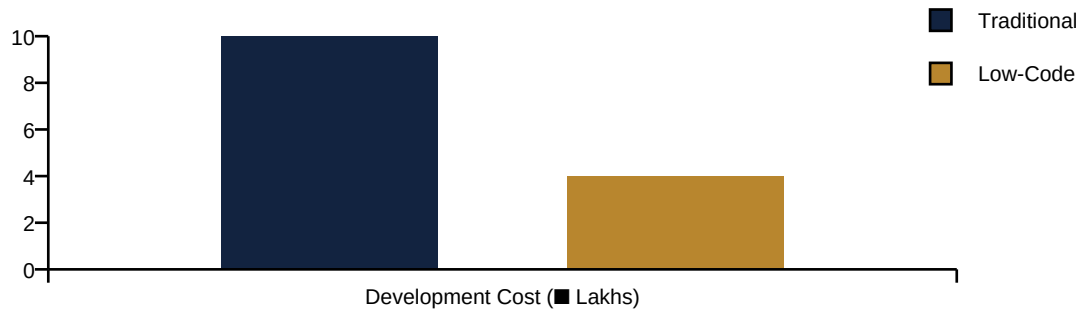

Figure 8.1 — Midpoint one-time development cost comparison.

8.1 Total Cost of Ownership (3-Year View)

Development cost alone understates the real financial gap. Low-code platforms carry recurring per-user or per-app licensing fees that traditional in-house systems do not, but traditional systems carry higher ongoing maintenance and infrastructure costs because every fix, upgrade, and integration must be engineered by hand. Over a typical three-year ownership window, low-code retains a lower total cost for colleges of small to medium size, though the gap narrows as user counts and customisation needs grow — licensing fees scale with usage in a way that in-house infrastructure costs do not.

9. Risk Analysis

Beyond time and cost, each approach carries a distinct risk profile that should inform the final decision.

Risk	Traditional Development	Low-Code Development
Vendor Lock-in	Low — source code is fully owned	Medium–High — tied to the platform vendor's roadmap and pricing
Skill Availability	Requires ongoing access to skilled developers	Broader pool, including citizen developers
Technical Debt	Accumulates with rushed manual coding	Lower, but constrained by platform limits
Scalability Risk	Low, if architected correctly	Moderate at very high transaction volumes

Risk	Traditional Development	Low-Code Development
Data Portability	Full control over migration	Depends on platform export/API capabilities
Long-Term Cost Creep	Maintenance cost can rise gradually	Licensing cost can rise with user growth

Table 9.1 — Key risk categories and how each approach is exposed.

10. Advantages & Disadvantages

10.1 Traditional Development

Advantages

- Complete customization of every feature and workflow
- Better performance for complex, high-transaction applications
- Greater long-term scalability across campuses and user bases
- Full control over source code, architecture, and data
- Suitable for large, multi-campus enterprise systems

Disadvantages

- Longer development time before go-live
- Higher upfront and maintenance cost
- Requires access to skilled, experienced developers
- Greater ongoing maintenance and bug-fixing effort

10.2 Low-Code Development

Advantages

- Significantly faster development and time-to-market
- Lower development cost with predictable licensing
- Easier ongoing maintenance through the visual platform
- Rapid, often one-click deployment to production
- Built-in security baselines and pre-built integrations
- Well suited to MVPs, pilots, and internal administrative tools

Disadvantages

- Customization limited by what the platform allows
- Platform dependency and vendor lock-in risk
- Recurring licensing costs that scale with usage
- Possible performance limitations for very large systems

11. Results of the Case Study

Metric	Traditional	Low-Code
Project Completion	5–6 months	1–2 months
Total Cost	High	Lower
Productivity	Moderate	High
Time-to-Market	Slow	Fast
Maintenance	Difficult	Easier
Best For	Large, highly customized systems	Small to medium applications and rapid delivery

Table 11.1 — Summary outcomes across the case study's evaluation criteria.

12. Decision Framework

The following quick-reference framework maps common institutional scenarios to the more suitable approach.

Scenario	Recommended Approach
Need a working system within 6–8 weeks	Low-Code
Limited in-house developer capacity	Low-Code
Standard admissions / attendance / fees workflow	Low-Code
Multi-campus system with heavy custom logic	Traditional
Deep integration with legacy ERP or SIS systems	Traditional
Need full ownership of source code and infrastructure	Traditional
Tight, fixed budget under ■5 Lakhs	Low-Code
Long-term system expected to scale to 50,000+ users	Traditional

Table 12.1 — Scenario-based guidance for choosing an approach.

13. Conclusion & Recommendation

For a College Management Application, both approaches are viable depending on project goals:

- **Traditional Development** is the better choice when the application requires extensive customization, integration with multiple external systems, and long-term scalability.
- **Low-Code Development** is ideal when the objective is rapid development, lower cost, and faster deployment with standard business features.

Recommendation

If the college needs a production-ready system quickly with common administrative features, **Low-Code Development** is generally the more efficient choice. If the institution requires unique workflows, complex integrations, or complete control over the application architecture, **Traditional Development** is the preferred approach. Many institutions ultimately adopt a hybrid path — launching an MVP on a low-code platform to validate workflows quickly, then re-platforming to a custom traditional build once scale and customisation needs are proven.

— End of Document —